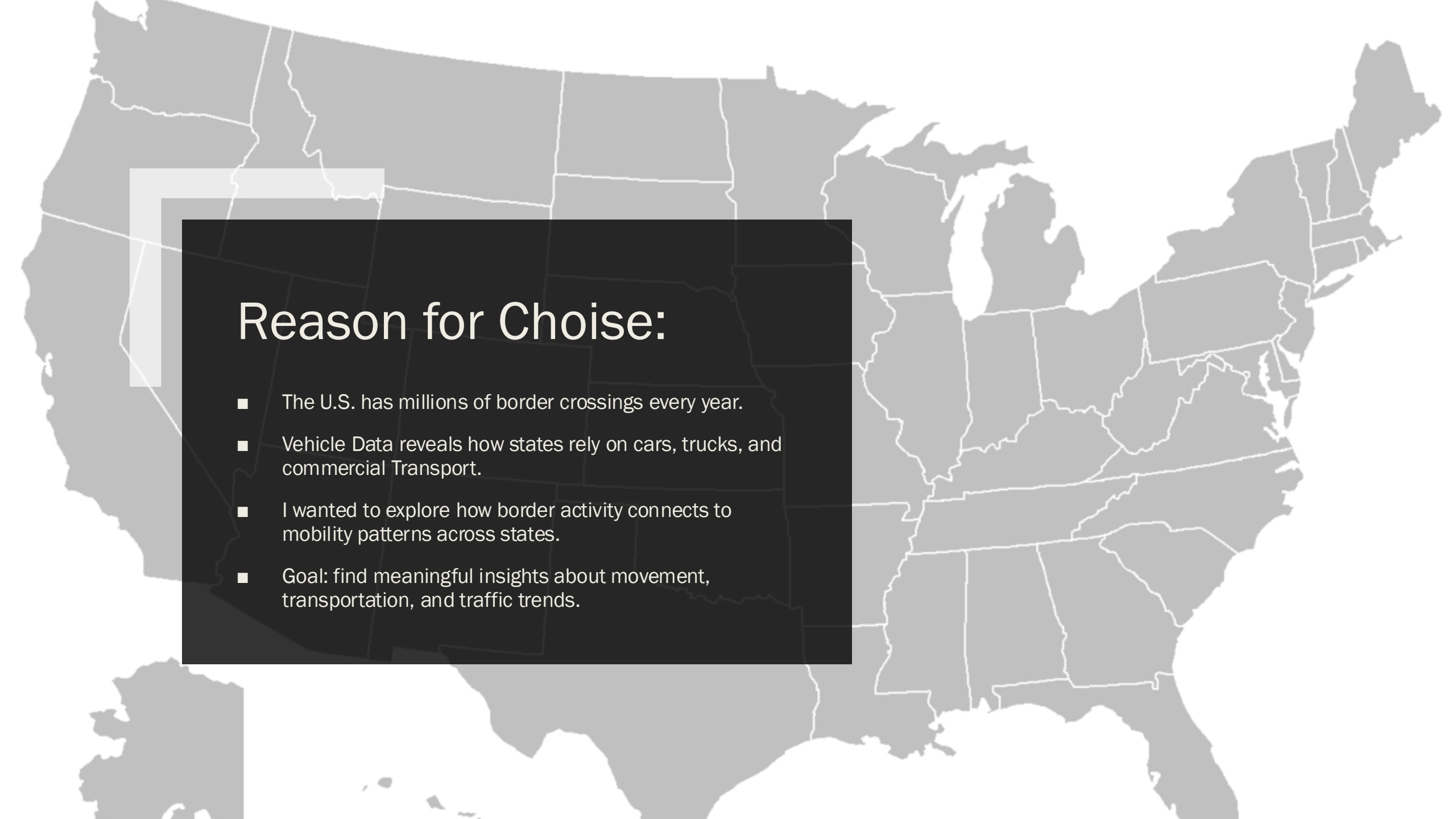




CS-2316: PHASE III

By: Ahmed Hassan





Reason for Choice:

- The U.S. has millions of border crossings every year.
- Vehicle Data reveals how states rely on cars, trucks, and commercial Transport.
- I wanted to explore how border activity connects to mobility patterns across states.
- Goal: find meaningful insights about movement, transportation, and traffic trends.

STATE	AUTOMOBILES			BUSES			TRUCKS			MOTORCYCLES			ALL MOTOR VEHICLES		
	PRIVATE AND COMMERCIAL (INCLUDING TAXICABS)	PUBLICLY OWNED	TOTAL	PRIVATE AND COMMERCIAL	PUBLICLY OWNED	TOTAL	PRIVATE AND COMMERCIAL	PUBLICLY OWNED	TOTAL	PRIVATE AND COMMERCIAL	PUBLICLY OWNED	TOTAL	PRIVATE AND COMMERCIAL	PUBLICLY OWNED	TOTAL
Alaska	132,017	2,282	134,299	7,883	579	8,462	498,305	12,421	510,726	25,627	11	25,638	663,832	15,293	679,125
Arizona	2,168,310	32,496	2,200,806	4,486	16,878	21,364	3,524,206	64,693	3,588,899	277,787	1,323	279,110	5,974,789	116,390	6,090,179
Arkansas (2)	910,969	17,987	928,956	10,017	823	10,840	2,132,989	45,167	2,178,156	98,364	—	98,364	3,152,339	63,977	3,216,316
California	13,558,261	237,848	13,796,109	28,802	67,163	95,965	16,092,736	331,803	16,424,539	789,100	13,400	802,500	30,468,899	650,214	31,119,113
Colorado	1,426,578	15,359	1,441,937	5,062	7,815	12,877	3,437,012	44,929	3,481,941	180,103	—	180,103	5,048,755	68,103	5,116,858
Connecticut (2)	1,100,313	420	1,100,733	9,951	24	9,975	1,589,239	4,600	1,593,839	84,876	—	84,876	2,784,379	5,044	2,789,423
Delaware	168,037	2,011	170,048	2,107	1,330	3,437	264,749	4,427	269,176	24,533	74	24,607	459,426	7,842	467,268
Dist. of Col.	188,795	12,412	201,207	761	5,876	6,637	152,531	15,881	168,412	4,201	344	4,545	346,288	34,513	380,801
Florida	7,664,561	98,807	7,763,368	24,115	35,658	59,773	11,026,555	163,565	11,190,120	647,360	2,841	650,201	19,362,591	300,871	19,663,462
Georgia	3,360,744	51,490	3,412,234	11,255	26,816	38,071	5,392,119	93,384	5,485,503	217,777	42	217,819	8,981,895	171,732	9,153,627
Hawaii	437,683	7,435	445,118	1,179	744	1,923	753,692	16,970	770,662	25,251	379	25,630	1,217,805	25,528	1,243,333
Idaho (3)	553,545	1,073	554,618	1,979	286	2,265	1,403,780	7,358	1,411,138	63,202	109	63,311	2,022,506	8,826	2,031,332
Illinois	3,726,086	4,357	3,730,443	36,719	3,910	40,629	6,251,801	20,023	6,271,824	291,478	61	291,539	10,306,084	28,351	10,334,435
Indiana	1,711,200	1,590	1,712,790	6,395	845	7,240	3,466,751	8,863	3,475,614	187,951	45	187,996	5,372,297	11,343	5,383,640
Iowa	1,055,910	9,626	1,065,536	3,598	5,337	8,935	2,484,491	25,341	2,509,832	194,789	330	195,119	3,738,788	40,634	3,779,422
Kansas	823,017	3,477	826,494	1,342	239	1,581	1,660,344	10,278	1,670,622	89,464	24	89,488	2,574,167	14,018	2,588,185
Kentucky	1,449,214	29,586	1,478,800	1,390	9,447	10,837	2,636,723	58,244	2,694,967	107,127	85	107,212	4,194,454	97,362	4,291,816
Louisiana	1,453,693	32,575	1,486,268	25,466	8,578	34,044	2,894,318	69,543	2,963,861	108,847	522	109,369	4,482,324	111,218	4,593,542
Maine	371,644	409	372,053	972	215	1,187	865,585	4,566	870,151	50,059	2	50,061	1,288,260	6,026	1,294,286
Maryland	1,997,970	26,812	2,024,782	6,865	9,025	15,890	2,728,346	46,429	2,774,775	103,526	81	103,607	4,836,707	82,347	4,919,054
Massachusetts	1,879,015	17,764	1,896,779	11,632	3,186	14,818	3,090,712	35,954	3,126,666	145,450	737	146,187	5,126,809	57,641	5,184,450
Michigan	2,679,646	37,415	2,717,061	1,364	866	2,230	6,329,922	97,987	6,427,909	255,703	805	256,508	9,266,635	137,073	9,403,708
Minnesota (2)	1,685,164	4,851	1,690,015	13,477	29	13,506	3,730,062	15,875	3,745,937	223,875	—	223,875	5,652,578	20,755	5,673,333
Mississippi	835,296	1,832	837,128	2,104	6,748	8,852	1,414,360	6,982	1,421,342	31,607	—	31,607	2,283,367	15,562	2,298,929
Missouri (2)	1,761,054	1,303	1,762,357	22,221	118	22,339	3,445,200	9,276	3,454,476	134,550	—	134,550	5,363,025	10,697	5,373,722
Montana	473,415	409	473,824	5,017	187	5,204	1,364,739	5,837	1,370,576	399,871	10	399,881	2,243,042	6,443	2,249,485
Nebraska	584,875	9,871	594,746	3,714	5,064	8,778	1,288,300	24,128	1,312,428	51,083	90	51,173	1,927,972	39,153	1,967,125
Nevada	1,008,913	10,909	1,019,822	306	6,503	6,809	1,545,446	21,429	1,566,875	76,951	1,934	78,885	2,631,616	40,775	2,672,391
New Hampshire	438,606	4,309	442,915	644	2,487	3,131	893,611	10,129	903,740	87,253	112	87,365	1,420,114	17,037	1,437,151
New Jersey (2)	2,327,099	23,560	2,350,659	21,715	2,453	24,168	3,431,741	44,465	3,476,206	148,634	—	148,634	5,929,189	70,478	5,999,667
New Mexico	585,077	19,000	604,077	2,164	4,631	6,795	1,156,344	46,283	1,202,627	56,502	379	56,881	1,800,087	70,293	1,870,380
New York (3)	2,955,805	12,436	2,968,241	77,460	5,295	82,755	5,286,843	37,519	5,324,362	736,000	4	736,004	9,056,108	55,254	9,111,362
North Carolina	3,231,937	44,133	3,276,070	10,384	22,426	32,810	5,403,249	83,615	5,486,864	199,812	350	200,162	8,845,382	150,524	8,995,906
North Dakota	234,910	3,881	238,791	1,180	2,337	3,517	789,328	15,778	805,106	46,074	21	46,095	1,071,492	22,017	1,093,509
Ohio	3,948,053	38,269	3,986,322	27,474	6,646	34,120	6,509,785	74,143	6,583,928	424,366	19	424,385	10,909,678	119,077	11,028,755
Oklahoma (2)	1,024,853	4,765	1,029,618	1,758	64	1,822	2,147,054	14,659	2,161,713	152,178	—	152,178	3,325,843	19,488	3,345,331
Oregon	1,337,439	17,915	1,355,354	9,192	8,192	17,384	2,584,315	41,003	2,625,318	140,681	596	141,277	4,071,627	67,706	4,139,333
Pennsylvania	3,696,871	41,883	3,738,754	52,112	2,756	54,868	6,616,552	86,709	6,703,261	371,184	762	371,946	10,736,719	132,110	10,868,829
Rhode Island	322,198	4,618	326,816	1,879	329	2,208	444,737	7,790	452,527	24,497	94	24,591	793,311	12,831	806,142
South Carolina	1,766,166	155,717	1,921,883	4,914	11,187	16,101	2,922,240	263,058	3,185,298	122,122	425	122,547	4,815,442	430,387	5,245,829
South Dakota	300,901	6,396	307,297	1,146	21,328	22,474	873,315	20,666	893,981	139,834	927	140,761	1,315,196	49,317	1,364,513
Tennessee	2,271,286	40,726	2,312,012	15,193	7,287	22,480	4,212,034	82,125	4,294,159	198,315	1,045	199,360	6,696,826	131,183	6,828,011
Texas	7,564,519	109,697	7,674,216	15,707	56,923	72,630	14,957,587	244,720	15,202,307	340,214	2,271	342,485	22,878,027	413,611	23,291,638
Utah	948,368	17,356	965,724	1,821	5,863	7,684	1,723,968	36,503	1,760,471	142,281	640	142,921	2,816,438	60,362	2,876,800
Vermont (2)	170,444	2,613	173,057	636	468	1,104	410,671	7,057	417,728	33,775	—	33,775	615,526	10,138	625,664
Virginia	2,898,548	40,111	2,938,659	8,041	27,918	35,959	4,519,646	77,055	4,596,701	191,235	733	191,968	7,617,470	145,817	7,763,287
Washington (2)	2,752,667	79,885	2,832,552	5,011	18,215	23,226	4,606,471	146,758	4,753,229	226,056	—	226,056	7,590,205	244,858	7,835,063
West Virginia (2)	465,489	8,157	473,646	154	53	207	1,108,647	21,939	1,130,586	47,577	880	48,457	1,621,867	31,029	1,652,896
Wisconsin (2)	1,707,309	22,679	1,729,988	13,484	56	13,540	3,604,984	52,979	3,657,963	279,926	256	280,182	5,605,703	75,970	5,681,673
Wyoming	183,725	383	184,108	1,101	108	1,209	672,102	4,211	676,313	28,646	9	28,655	885,574	4,711	890,285
Total	98,293,442	1,372,935	99,666,377	526,607	431,448	958,055	169,671,101	2,692,977	172,364,078	9,153,484	32,772	9,186,256	277,644,634	4,530,132	282,174,766

Datasets

- Dataset 1 (CSV): U.S. Border Crossing
- Dataset 2 (JSON): NYC Bike Stations
- Dataset 3 (Webscrapping): Vehicle Registrations by Station

Data Cleaning (Dataset 1 (CSV): U.S. Border Crossing)

- Purpose: To clean and prepare the official U.S. Border Crossing Entry Data for use in insights and models.
- Loaded the CSV containing border crossing records from 1996–2025.
- Identified duplicated rows and removed them (20 duplicates).
- Saved the cleaned dataset as Cleaned_Border_Crossing_Data.csv.
- This makes calculations much more reliable since rows aren't counted twice.

```
import pandas as pd
def data_parser():
    df = pd.read_csv("Border_Crossing_Entry_Data.csv")
    duplicates = df[df.duplicated(keep = False)]
    print(f"The total number of duplicated rows are {len(duplicates)}.")
    df_cleaned = df.drop_duplicates()
    df_cleaned.to_csv("Cleaned_Border_Crossing_Data.csv")
    return df_cleaned

##### Function Call #####
data_parser()
```

The total number of duplicated rows are 20.

Dataset 2 (JSON): NYC Bike Stations

```
import requests
import pandas as pd
import numpy as np
import json

def web_parser1(url):
    data = requests.get(url).json()
    df = pd.DataFrame(data["data"]["stations"])
    df.fillna(0, inplace = True)
    df.to_csv("Cleaned_Stations_JSON.csv")
    return df

# ##### Function Call #####
web_parser1("https://gbfs.citibikenyc.com/gbfs/en/station_status.json")
```

- Purpose: To collect live NYC bike station data and convert it into a clean CSV for analysis.
- Pulled JSON data directly from the CitiBike API.
- Extracted the stations section and converted it into a DataFrame.
- Filled missing values to avoid errors in later calculations.
- Saved the cleaned dataset as Cleaned_Stations_JSON.csv.
- By replacing NaN's with 0's it makes any calculation much more reliable when comparing data.

Dataset 3 (Web scraping): Vehicle Registrations by Station

- Purpose: To scrape automobile and truck counts for each U.S. state from a federal website.
- Used Requests + BeautifulSoup to download and parse the HTML table.
- Extracted header + subheader rows and standardized the first column as "STATE."
- Cleaned numeric values (removed commas, replaced missing values with the column average).
- Saved the cleaned output as cleaned_web2.csv.
- This process will make the table complete and ready for any calculation or comparison that needs to be made.

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

def web_parser2(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text)

    table = soup.find("table")
    all_rows = table.find_all("tr")
    data = []
    for row in all_rows:
        cells = row.find_all(['th', 'td'])
        cell_values = [cell.get_text(strip = True) for cell in cells]
        data.append(cell_values)

    header = data[0]
    subheader = data[1]
    subheader.insert(0, "STATE")
    rows = data[2:]

    wanted_column = 11
    number_values = []
    for row in rows:
        value = row[wanted_column]
        if value.strip() != "-":
            clean_value = value.replace(",", "")
            if clean_value.isdigit():
                number_values.append(int(clean_value))
    average = sum(number_values) // len(number_values)

    for row in rows:
        if row[wanted_column].strip() == "-":
            row[wanted_column] = average

    final = [header, subheader] + rows
    df = pd.DataFrame(final)
    df.to_csv("cleaned_web2.csv")
    return df

##### Function Call #####
web_parser2("https://www.fhwa.dot.gov/policyinformation/statistics/2022/mv1.cfm#foot2")
```

Insight 1 & Visualization 1

- Load the cleaned border crossing dataset and the cleaned vehicle dataset.
- Extract each state's truck counts from the vehicle dataset.
- Group the border dataset by state and sums total border crossings per state.
- Merge both datasets so each state has Total Trucks and Total Crossings.
- Fit a linear regression model to predict crossings from truck counts.
- Return a dataset with actual + predicted values.
- Significance: This analysis explores whether states with more truck activity also have higher border movement. Since trucks drive much of commercial freight, finding a correlation suggests economic or transportation patterns.

```
def insight1():
    border_df = pd.read_csv("Cleaned_Border_Crossing_Data.csv")
    vehicles_df = pd.read_csv("cleaned_web2.csv")

    vehicles_data = vehicles_df.iloc[2:, :]
    # print(vehicles_data)
    state_col = vehicles_data.iloc[:, 1]
    trucks_col = vehicles_data.iloc[:, 9]

    vehicles_clean = pd.DataFrame({"STATE": state_col, "TRUCKS": trucks_col})
    vehicles_clean["TRUCKS"] = (vehicles_clean["TRUCKS"].astype(str).str.replace(",", "").astype(float))
    border_state = (border_df.groupby("State")["Value"].sum().reset_index().rename(columns={"Value": "Total Crossings"}))

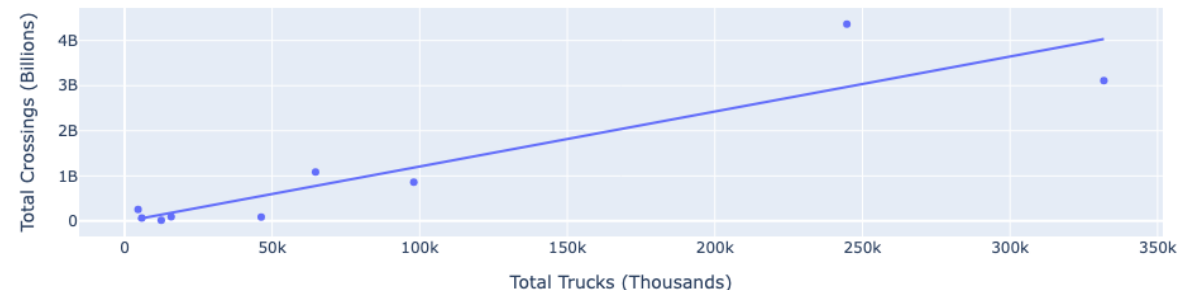
    merged = border_state.merge(vehicles_clean, left_on="State", right_on="STATE", how="inner")
    # print(merged)
    X = merged[["Total Crossings"]]
    y = merged["TRUCKS"]

    model = LinearRegression()
    model.fit(X, y)
    predictions = model.predict(X)

    merged["Predicted"] = predictions
    return (merged.head(10))

##### Function Call #####
insight1()
```

Relationship Between Border Crossings and State Truck Counts



```
def insight2():
    df = pd.read_csv("cleaned_web2.csv")
    df2 = df.iloc[2:, :]
    states = df2.iloc[:, 1]
    auto = df2.iloc[:, 3].astype(str).str.replace(",", "").astype(float)
    trucks = df2.iloc[:, 9].astype(str).str.replace(",", "").astype(float)

    X = np.column_stack([auto, trucks])
    k = 3
    rng = np.random.RandomState(42)
    initial_centroids = rng.permutation(X.shape[0])[:k]
    centroids = X[initial_centroids]
    for i in range(5):
        labels = pairwise_distances_argmin(X, centroids)
        new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])
        centroids = new_centroids

    result = pd.DataFrame({"State": states, "Autos": auto, "Trucks": trucks, "Cluster": labels})
    return result.head(10)

##### Function Call #####
insight2()
```

	State	Autos	Trucks	Cluster
2	Alaska	2282.0	12421.0	1
3	Arizona	32496.0	64693.0	1
4	Arkansas (2)	17987.0	45167.0	1
5	California	237848.0	331803.0	0
6	Colorado	15359.0	44929.0	1
7	Connecticut (2)	420.0	4600.0	1
8	Delaware	2011.0	4427.0	1
9	Dist. of Col.	12412.0	15881.0	1
10	Florida	98807.0	163565.0	0

Insight 2

- Load vehicle data for all states from the scraped table.
- Clean the numeric columns (removes commas and converts to numbers).
- Build a dataset containing each state's automobile count and truck count.
- Used K-Means clustering to group states into 3 clusters based on their vehicle profiles.
- Return the dataset with each state assigned to a cluster.
- Significance: This insight shows which states behave similarly in terms of vehicle usage. It helps identify clusters like: High, average, and low activity states. These groupings help us understand national transportation patterns.

Insight 3

- Load the cleaned NYC bike JSON data.
- Calculate total bikes available and the average number of bikes per station.
- Load the cleaned border crossing dataset.
- Filter for Pedestrian crossing rows and sums all pedestrian crossings nationwide.

- Return a small summary table with three values:

- *Total NYC bikes*
- *Average bikes per station*
- *Total U.S. pedestrian border crossings*

```
def insight3():  
    json_df = pd.read_csv("Cleaned_Stations_JSON.csv")  
    border_df = pd.read_csv("Cleaned_Border_Crossing_Data.csv")  
    total_city_bikes = json_df["num_bikes_available"].sum()  
    avg_bikes_station = json_df["num_bikes_available"].mean()  
  
    pedestrian_df = border_df[border_df["Measure"].str.contains("Pedestrian")]  
    total_pedestrian_crossings = pedestrian_df["Value"].sum()  
  
    final = pd.DataFrame({"Total NYC Bikes": [total_city_bikes], "Average Bikes / Station": avg_bikes_station,  
                          "Total Border Pedestrian Crossings": total_pedestrian_crossings})  
    return final  
  
##### Function Call #####  
insight3()
```

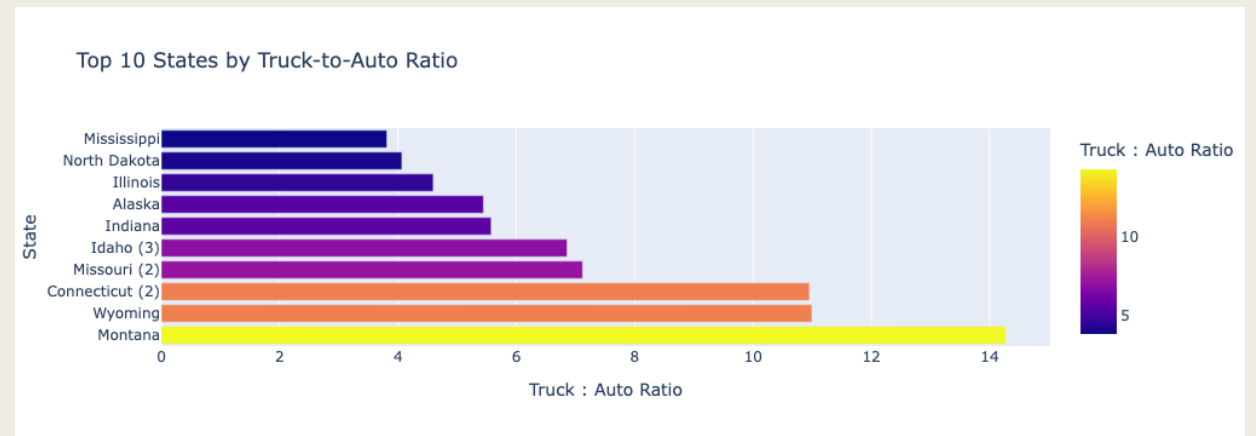
	Total NYC Bikes	Average Bikes / Station	Total Border Pedestrian Crossings
0	29009	12.568891	1279319340

- Significance: This insight compares a dense, urban mobility system (NYC bikes) with national pedestrian border travel. It highlights how much larger federal border flow is compared to a single city's mobility system.

Insight 4 & Visualization 2

- Loads the cleaned vehicle dataset from the FHWA scrape.
- Extracts automobile and truck counts.
- Computes a truck-to-auto ratio for each state by dividing trucks by cars.
- Sorts states from highest to lowest ratio. Returns the top 10 states with the highest ratios.
- Significance: Some states have extremely high truck usage compared to automobiles. These states likely depend heavily on commercial freight routes or rural transportation networks. Showing the top 10 highlights where trucking dominates personal vehicle travel.

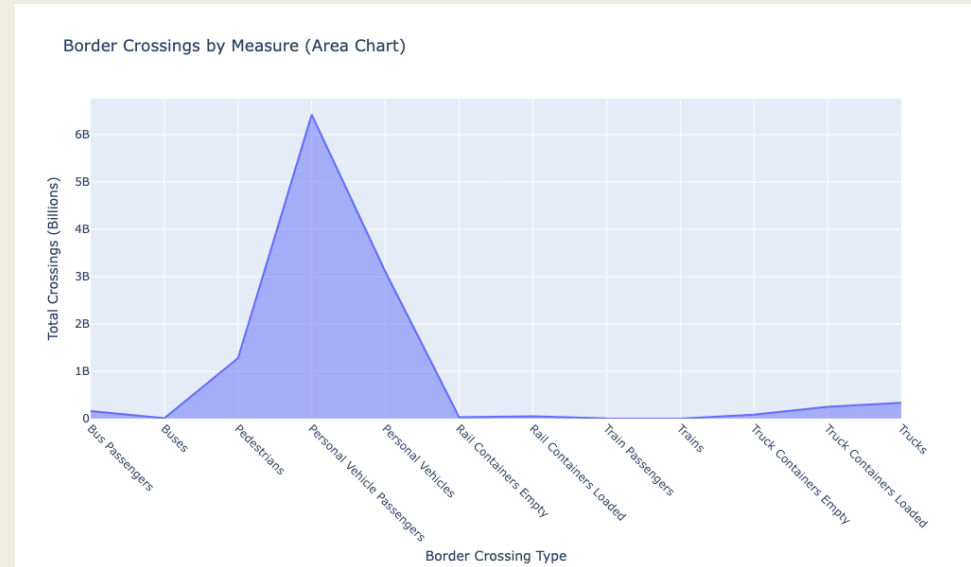
```
def insight4():  
    df = pd.read_csv("cleaned_web2.csv")  
    df2 = df.iloc[2:, :]  
    states = df2.iloc[:, 1].values  
    autos = df2.iloc[:, 3].astype(str).str.replace(",", "").astype(float)  
    trucks = df2.iloc[:, 9].astype(str).str.replace(",", "").astype(float)  
    ratio = trucks / autos  
  
    final = pd.DataFrame({ "State": states, "Automobiles": autos, "Trucks": trucks,  
                           "Truck-to-Auto Ratio": ratio})  
    final_sorted = final.sort_values("Truck-to-Auto Ratio", ascending = False)  
    return final_sorted.head(10)  
  
##### Function Call #####  
insight4()
```



Insight 5 & Visualization 3

- Loads the cleaned border crossing dataset.
- Groups all rows by the Measure (transportation type).
- Sums total crossings for each category.
- Returns a summary table with totals for buses, pedestrians, personal vehicles, trains, etc.
- Significance: This insight breaks down mobility by transportation type. It clearly shows that personal vehicle passengers dominate border crossings, followed by pedestrian traffic. Rail, bus, and commercial containers are much smaller contributors.

```
def insight5():  
    df = pd.read_csv("Cleaned_Border_Crossing_Data.csv")  
    final = (df.groupby("Measure")["Value"].sum().reset_index())  
    return final  
  
##### Function Call #####  
insight5()
```



Conclusion

- I found strong ties between truck availability and border movement
- Significant differences in transportation reliance across states, and clear dominance of personal vehicle crossings.
- I learned a lot about clustering, regression, merging datasets, and building effective visualizations.
- The biggest challenge was cleaning the datasets and making them match for analysis.

50 U.S. STATES (USPS)

Alabama (AL)
Alaska (AK)
Arizona (AZ)
Arkansas (AR)
California (CA)
Colorado (CO)
Connecticut (CT)
Delaware (DE)
Florida (FL)
Georgia (GA)
Hawaii (HI)
Idaho (ID)
Illinois (IL)
Indiana (IN)
Iowa (IA)
Kansas (KS)
Kentucky (KY)
Louisiana (LA)
Maine (ME)
Maryland (MD)
Massachusetts (MA)
Michigan (MI)
Minnesota (MN)
Mississippi (MS)
Missouri (MO)
Montana (MT)
Nebraska (NE)
Nevada (NV)
New Hampshire (NH)
New Jersey (NJ)
New Mexico (NM)
New York (NY)
North Carolina (NC)
North Dakota (ND)
Ohio (OH)
Oklahoma (OK)
Oregon (OR)
Pennsylvania (PA)